Lecture 4

11 October 2026 23:13

The reason why we get the gibblish output cause, we are feeding the last character only, we have to train the whole model using every last letters or previous token

----------the actual code----------

```
# 1. Create the optimizer (AdamW is the standard modern optimizer)
optimizer = torch.optim.AdamW(m.parameters(), lr=1e-3)

batch_size = 32

# 2. Train for 10,000 steps
for steps in range(10000):

    # Grab a fresh batch of training data
    xb, yb = get_batch('train')

    # Forward pass: evaluate the loss
    logits, loss = m(xb, yb)

    # --- The 3 Golden Steps of PyTorch Training ---
    optimizer.zero_grad(set_to_none=True)  # Step A: Reset previous gradients
    loss.backward()                        # Step B: Calculate new gradients (backprop)
    optimizer.step()                       # Step C: Update model parameters

    # Print the loss occasionally to see progress
    if steps % 1000 == 0:
        print(f"Step {steps}: loss {loss.item():.4f}")
```

The "3 Golden Steps" of Every PyTorch Loop

Every time you train a neural network in PyTorch, you will always see these three lines in order:

1. Optimizer.zero_grad(set_to_none=True)
2. Loss.backward()
3. Optimizer.step()

1. optimizer.zero_grad(set_to_none=True)

1. By default, PyTorch **accumulates (adds up)** gradients between iterations.
2. If you don't reset them, gradients from step 2 will add on top of step 1, corrupting the math!
3. zero_grad() wipes the slate clean before computing gradients for the new batch.

   Hence, every new batch will have new weights

V IMP

2. loss.backward() (Backpropagation)

1. PyTorch walks backward through the computational graph from the loss all the way to m.token_embedding_table.
2. It calculates the **gradient** for every number in the embedding table (i.e., *"should this number increase or decrease to make the loss smaller?"*).

As we saw in the before lectures, backward is the back propogation, which is imp, as it reduce the loss, by seeing it that the loss is been produce, so this step is required

3. optimizer.step()

- The optimizer (AdamW) looks at those gradients and takes a step, updating all the numbers in the embedding table in the direction that lowers the loss.

Basically what is happening that, the loss and the output is generated due to the gradients which are been trained in the embedding table, but initially it's been trained with very random numbers, therefore the loss is high and the l=output is not what is required.

Hence, AdamW helps to train the embedding table by introducing new weights in the table ----------> we can see this in detail in the neural network

The Tiny Correction:

AdamW does **NOT introduce new weights**. The embedding table always has the **same fixed number of weights** (65 x 65 = 4,225 numbers throughout the entire training).
What AdamW actually does is **nudge / adjust the existing weights** in the right direction!

Think of it this way:

Imagine the embedding table is a **wall of 4,225 knobs** (like a mixing board in a music studio).

Stage	What happens
Initialization	All 4,225 knobs are set to **random values**. Output = noise. Loss = High.
loss.backward()	PyTorch calculates for **each knob**: *"Turn left or turn right to reduce the loss?"* (This is the gradient.)
optimizer.step()	AdamW **turns each knob slightly** in the correct direction.
Repeat 10,000 times	After thousands of small adjustments, all 4,225 knobs are tuned to produce good Shakespeare text.

So in one clean sentence:

AdamW does not **add** new weights. It **repeatedly fine-tunes the same existing weights** (starting from random numbers) until the embedding table produces low-loss, sensible predictions.

Step 1 : why do we create optimizer and what it is ?

An **optimizer** is an algorithm that answers one specific question after every training step:

*"We know the current weights are wrong (because loss is high). We know the direction each weight should move (because loss.backward() calculated gradients). But by exactly **how much** should each weight move?"*

The optimizer decides the **size and smartness of each weight update**.

1. Assume weigh like the number of random numbers or inputs given to get answer in the neural network,
2. and the gradient is the loss which let say needs to be nearly around zero, so that if gradient is positive i.e. loss is high then the weight which are causing it should decrease, similalry,
3. If the gradient is in negative, i.e. loss is in negative still the weights should be increase to minimum the loss.

Why do we need it? The Problem it Solves.

After loss.backward(), every weight in the model has a **gradient** — a number that says:

1. **Positive gradient (+):** This weight should decrease to lower the loss.
2. **Negative gradient (-):** This weight should increase to lower the loss.

V IMP:
But here is the problem:
If we just blindly follow the gradient, we can overshoot the correct value!

Meaning : if we are getting a right answer, but due to dependency on the gradient, as the gradient which only tell the loss should be minimum and resp the weights should be manage, leads to overshooting the wright answer.

For example:

Imagine you are rolling a ball down a hilly landscape trying to find the lowest valley (the minimum loss):

text

```
      \    /  \    /
       \  /    \  /
        \/      \/
Good    ^
```

Valley | Bad Valley
START

START

1. **Too Large a Step:** The ball rolls so fast it flies over the good valley and lands in a worse spot!
2. **Too Small a Step:** The ball moves so slowly training takes forever.
3. **Smart Step:** The optimizer finds the right step size to land near the lowest valley efficiently.

This "landscape" is called the **Loss Landscape**, and finding the lowest point is called **Optimization**.

1. What does AdamW specifically do?

AdamW (Adaptive Moment Estimation with Weight Decay) is the **most popular modern optimizer** in deep learning. It is smarter than basic gradient descent in two key ways:

1. Adaptive Learning Rates (The "Adam" part)

Instead of using the same step size for every weight, AdamW gives **each weight its own personal step size**:

1. Weights that haven't changed much recently → **Take a bigger step** (catch up faster).
2. V IMP:

Weights that are changing rapidly and erratically → **Take a smaller step** (be careful, don't overshoot).

V.IMP

2. Weight Decay (The "W" part)

AdamW adds a small penalty that **pushes weights toward zero** if they are becoming too large0

.

1. V IMP:

This prevents the model from becoming overconfident on specific patterns

.

1. It improves generalization (the model works better on new, unseen text).

The lr (Learning Rate) Parameter:

python
```
optimizer = torch.optim.AdamW(m.parameters(), lr=1e-3)
```

1. **m.parameters()** tells AdamW: *"Here are all the weights you are responsible for updating"* (in our case, the 4,225 numbers in the embedding table).
2. **lr=1e-3** (Learning Rate = 0.001) is the **baseline step size** — it controls roughly how big each update is.

Learning Rate	Effect	Risk
Too high (e.g., 0.1)	Updates are huge, trains fast initially	Overshoots, loss bounces around, never converges
Too low (e.g., 0.000001)	Updates are tiny, very precise	Training takes forever
Just right (1e-3)	Smooth, steady descent toward low loss	This is the sweet spot for most models

Summary :

1. **Optimizer:** An algorithm that takes the gradients calculated by loss.backward() and uses them to update (nudge) the model weights in the direction that reduces loss.
2. **AdamW:** The modern standard optimizer that gives each weight its own adaptive step size and adds weight decay to prevent overfitting.
3. **Learning Rate (lr):** Controls how large each update step is. Too large = overshooting. Too small = too slow.
4. **m.parameters():** Tells the optimizer which weights it is responsible for managing (the embedding table in our Bigram model).

V IMP
Why we are using the batch size as 32 now ? → IMP

The batch size was **never changed for correctness** — it was changed purely for **training efficiency and stability**.

Why a Bigger Batch Size During Training?

1. Better Gradient Estimates (Less Noisy Updates)

When you compute the loss and gradients from just **4 sequences**, you are making decisions based on a very small sample. The gradient might be noisy and point in a slightly wrong direction.
When you use **32 sequences**, you are averaging the gradient signal across 32 different text chunks. This gives a **much more reliable, smoother gradient estimate** before updating the weights.

It's like this :

text
Batch size 4: Average of 4 opinions --> Noisy, unreliable direction
Batch size 32: Average of 32 opinions --> Stable, trustworthy direction

2. GPU Efficiency



Modern GPUs are built to process large matrix operations **in parallel**.
Sending 32 sequences at once keeps the GPU **fully busy**:

- B = 4 → GPU is 80% idle, 20% working. Wasteful!
- B = 32 → GPU is nearly fully utilized. Much faster per second!

3. Why Not Use Batch Size = 10,000 then?

- **Memory:** Each sequence takes up RAM/GPU memory. Too large a batch crashes with an **Out of Memory** error.
- **Diminishing Returns:** Beyond a certain size, the gradient estimate doesn't improve much but memory cost keeps growing.

Summary :

Batch Size	Gradient Quality	GPU Efficiency	Memory Cost
Too Small (4)	Noisy, unreliable	Poor (GPU idle)	Very Low
Just Right (32)	Smooth, stable	Good	Moderate
Too Large (1024+)	Very smooth	Excellent	Very High (can OOM)

So the code will be this:

```
# 1. Create the optimizer (AdamW is the standard modern optimizer)
optimizer = torch.optim.AdamW(m.parameters(), lr=1e-3)

batch_size = 32

# Train for 10,000 steps
for steps in range(10000):

    # sample a batch of data
    xb, yb = get_batch("train")

    # Forward pass: evaluate the loss
    logits, loss = m(xb, yb)

    # --- The 3 Golden Steps of PyTorch Training ---
    optimizer.zero_grad(set_to_none=True)  # Re-set previous gradients
    loss.backward()                        # Calculate new gradients (backprop)
    optimizer.step()                       # Update model parameters

# this is the final loss generated
print(loss.item())
```

Output:
2.3781542778015137

Another code needed to get the new text

```
#print(decode(m.generate(idx = torch.zeros((1, 1), dtype=torch.long), max_new_token=100)[0].tolist()))

generate_tokens = m.generate(idx = torch.zeros((1, 1), dtype=torch.long), max_new_token=500)
print(decode(generate_tokens[0].tolist()))
```

Output:

pu wotif$w brurot pinou banks y whocois arioofunce trd thir omiththio tas ofe, aly d beanerok GSclaves.
Y shay wacometend hy, s.

Whel ove bu grdde.
wolke IO: ane lde
ne m IOrduranars f lcallKe ig, hay bagithabemendin tou me, t fieo3wathearar wo--ds m as ag Ighowa ton
smusissad!Thald h

INThangr, st tchirin later

IUMa hedim,
Pe int ongelorurdo misin the tseld

Hence, we can see the model Is making progress, but it not all correct the way we want, cause, till now we are not trainig the model with its past tokens also,

We are taking only the last token.